ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Đồ án 2 - Khai Thác Tập Phổ Biến

Nghiên cứu, Cài đặt, và Đánh giá

Nguyễn Đỗ Bảo

Trường Đại học Khoa học Tự nhiên

Khoa Công nghệ Thông tin

CSC14004 - Khai thác Dữ liệu và Ứng dụng

Hồ Chí Minh, Tháng 4/2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Đồ án 2 - Khai Thác Tập Phổ Biến

Nghiên cứu, Cài đặt, và Đánh giá

Nguyễn Đỗ Bảo

MSSV: 23127026

Supervisor: Lê Nhựt Nam
Khoa Công nghệ Thông tin

Trường Đại học Khoa học Tự nhiên

Khoa Công nghệ Thông tin

CSC14004 - Khai thác Dữ liệu và Ứng dụng

Hồ Chí Minh, Tháng 4/2026

Mục lục

Danh sách hình vẽ ii

Danh sách bảng 1

1 Nền tảng lý thuyết **2**

1.1 Bài toán khai thác tập phổ biến 2

1.1.1 Cơ sở dữ liệu giao dịch (transaction database) 2

1.1.2 Độ hỗ trợ (support) 2

1.1.3 Tập phổ biến (frequent itemset) 2

1.1.4 Tập đóng (closed itemset) và tập tối đại (maximal itemset) . . . 3

1.1.5 Tính chất Apriori (downward closure) 3

1.2 Phân tích thuật toán AprioriTID 3

1.2.1 Ý tưởng cốt lõi 4

1.2.2 Cấu trúc dữ liệu chính 4

1.2.3 Mã giả (Pseudocode) 5

1.2.4 Phân tích độ phức tạp 5

1.2.5 So sánh lịch sử 9

2 Ví dụ minh họa **10**

2.1 Ví dụ 1: Cơ sở 10

2.2 Ví dụ 2: Tình huống đặc biệt 11

3 Thực nghiệm và đánh giá **14**

Appendices

Danh sách hình vẽ

1.1 Thuật toán AprioriTID tìm tập phổ biến dựa trên sinh ứng viên. 6

2.1 Ví dụ 1 — Các bước thực thi AprioriTID trên CSDL 5 giao dịch với minsup = 2. 10
2.2 Ví dụ 2 — Bước $k = 1$: Tất cả 7 item đều phổ biến. 11
2.3 Ví dụ 2 — Bước $k = 2$: $\overline{C}_2$ có 60 entries, gấp 2.5 lần CSDL gốc (24 entries). . 12

Danh sách bảng

1.1 Các ký hiệu dùng trong phân tích độ phức tạp. 7
1.2 Độ phức tạp không gian của các cấu trúc chính. 8

1

Nền tảng lý thuyết

1.1 Bài toán khai thác tập phổ biến

1.1.1 Cơ sở dữ liệu giao dịch (transaction database)

Cơ sở dữ liệu giao dịch (CSDL) $\mathcal{D}$ bao gồm một tập hợp các giao dịch T, trong đó mỗi giao dịch chứa một tập hợp các hạng mục không rỗng (*itemset*) $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ sao cho $T \subseteq \mathcal{I}$ và được gán với một định danh giao dịch duy nhất (TID). Ta nói giao dịch T chứa X (với X là tập các hạng mục trong $\mathcal{I}$), nếu $X \subseteq T$.

1.1.2 Độ hỗ trợ (support)

Độ hỗ trợ (*support*) của một tập hạng mục X là phần trăm số lượng giao dịch có chứa X trong cơ sở dữ liệu giao dịch $\mathcal{D}$, có thể được định nghĩa bằng (1.1):

$$sup(X) = \frac{|\{T \mid X \subseteq T \wedge T \in \mathcal{D}\}|}{|\mathcal{D}|} \tag{1.1}$$

Có hai cách để biểu diễn độ hỗ trợ là **độ hỗ trợ tuyệt đối** (*absolute support* hay *support count*) và **độ hỗ trợ tương đối** (*relative support*).

Độ hỗ trợ tuyệt đối của một tập hạng mục X là số lượng giao dịch trong CSDL chứa X, được định nghĩa bằng:

$$support_count(X) = |\{T \mid X \subseteq T \wedge T \in \mathcal{D}\}| \tag{1.2}$$

Độ hỗ trợ tương đối của một tập hạng mục X là tỷ lệ giữa độ hỗ trợ tuyệt đối của X và tổng số giao dịch trong CSDL, biểu diễn bằng (1.1).

1.1.3 Tập phổ biến (frequent itemset)

Ta nói X là **tập phổ biến** khi $sup(X) \geq minsup$ trong đó *minsup* là ngưỡng hỗ trợ tối thiểu được định trước. Một tập hạng mục chứa k hạng mục là một **k-itemset**. Tập hợp

của các tập phổ biến k-itemset được ký hiệu là L_k.

1.1.4 Tập đóng (closed itemset) và tập tối đại (maximal itemset)

Tập hạng mục X được gọi là **đóng** (*closed*) trong tập dữ liệu D nếu không tồn tại một siêu tập hợp (*superitemset*) Y (tức là, mọi phần tử của X đều thuộc Y, nhưng tồn tại ít nhất một phần tử của Y không thuộc X) nào chứa X mà Y có cùng số hỗ trợ (*support count*) với X trong D, hay nói cách khác:

$$\nexists Y \mid (X \subset Y) \wedge (sup(X) = sup(Y)) \tag{1.3}$$

Tập hạng mục X là **tập phổ biến đóng** (*closed frequent itemset*) khi thỏa mãn cả tính đóng và tính phổ biến.

Tập hạng mục X được gọi là **tập tối đại** (*maximal itemset*) trong tập dữ liệu D nếu không tồn tại một siêu tập hợp Y nào chứa X trong D, hay nói cách khác:

$$\nexists Y \mid (X \subset Y) \tag{1.4}$$

Tập hạng mục X là **tập phổ biến tối đại** (*maximal frequent itemset*) khi thỏa mãn cả tính tối đại và tính phổ biến.

Mối quan hệ giữa tập phổ biến (FI), tập phổ biến đóng (CFI), và tập phổ biến tối đại (MFI) được biểu diễn như sau: MFI $\subseteq$ CFI $\subseteq$ FI.

1.1.5 Tính chất Apriori (downward closure)

Tính chất Apriori: Tất cả các tập con không rỗng của một tập hạng mục phổ biến cũng phải là các tập phổ biến.

Tính chất đơn điệu giảm (antimonotonicity): Nếu một tập hợp không vượt qua được một phép thử, thì tất cả các tập hợp cha của nó cũng sẽ không vượt qua được phép thử đó.

Chứng minh: Nếu một tập hạng mục I không thỏa mãn điều kiện về độ hỗ trợ tối thiểu (*minsup*), thì I sẽ không phải là tập phổ biến (tức là $P(I) < minsup$). Khi đó, nếu có một hạng mục a được thêm vào I, thì tập hạng mục $(I \cup \{a\})$ sẽ không thể xuất hiện nhiều lần hơn I. Vả lại, có thể coi $(I \cup \{a\})$ là tập cha của I (vì $I \subset (I \cup \{a\})$). Do đó, $(I \cup \{a\})$ cũng không thể là tập phổ biến (do $P(I \cup \{a\}) < minsup$).

1.2 Phân tích thuật toán AprioriTID

Thuật toán tìm tập hạng mục lớn (*large itemsets*) hoạt động lặp đi lặp lại qua nhiều lượt quét CSDL. Ở lượt đầu tiên, thuật toán đếm độ hỗ trợ (*support*) của từng hạng mục để tìm các large 1-itemset. Ở các vòng tiếp theo, các large itemset từ vòng trước được dùng làm cơ sở sinh ra các tập ứng viên (*candidate itemsets*). Thuật toán tiếp tục quét

CSDL để đếm support cho ứng viên, lọc ra các large itemset mới và lặp lại quá trình này cho đến khi không tìm thêm được tập nào nữa.

1.2.1 Ý tưởng cốt lõi

Thuật toán Apriori tìm tập phổ biến thông qua quá trình sinh ứng viên và kiểm tra theo từng mức độ dài, trong đó các large k-itemset được sử dụng để khám phá các large (k+1)-itemset. Ý tưởng cốt lõi của AprioriTID dựa trên một biến thể tối ưu hóa gọi là giảm thiểu giao dịch (*transaction reduction*). Nếu một giao dịch không chứa bất kỳ k-itemset phổ biến nào thì nó cũng không thể chứa bất kỳ (k+1)-itemset phổ biến nào khác. Do đó, ta loại bỏ các TID này và không cần xét đến chúng trong các lần quét CSDL ở các vòng lặp tiếp theo. Thay vì phải quét lại CSDL gốc nhiều lần, AprioriTID tạo ra một tập dữ liệu mã hóa trung gian, ký hiệu là $\overline{C}_k$ để lưu vết những ứng viên có mặt trong từng giao dịch. Ở các vòng lặp sau, tập $\overline{C}_k$ này có thể trở nên nhỏ hơn rất nhiều so với kích thước của CSDL gốc, giúp tiết kiệm đáng kể chi phí đọc đĩa (I/O).

Đầu tiên, tìm tập hợp 1-itemset phổ biến bằng cách đếm số lần xuất hiện của TID chứa 1-itemset, và thu thập những 1-itemset thỏa mãn minsup. Tập hợp kết quả được ký hiệu là L_1. Tiếp theo, L_1 được sử dụng để tìm L_2, tập hợp các 2-itemset phổ biến, được sử dụng để tìm L_3, và cứ thế tiếp tục cho đến khi không còn k-itemset phổ biến nào được tìm thấy nữa.

Các thuật toán đi trước như AIS hay SETM sinh các tập ứng viên ngay tức thì (on-the-fly) bằng cách đọc từng giao dịch và mở rộng các tập phổ biến có trong giao dịch đó. Cách này tạo ra vô số ứng viên dư thừa, dẫn đến lãng phí tài nguyên. AprioriTID khắc phục điều này bằng cách sinh ứng viên một cách độc lập thông qua việc nối các tập phổ biến bước trước, tạo ra tập ứng viên nhỏ hơn rất nhiều.

1.2.2 Cấu trúc dữ liệu chính

AprioriTID sử dụng một cấu trúc dữ liệu mảng tuần tự đặc trưng là tập $\overline{C}_k$. Mỗi phần tử trong $\overline{C}_k$ đại diện cho một giao dịch, có định dạng: $\langle \text{TID}, \{X_k\} \rangle$, trong đó X_k là k-itemset xuất hiện trong TID. Nếu một giao dịch không chứa bất kỳ ứng viên nào, nó sẽ không có mục (*entry*) tương ứng trong $\overline{C}_k$.

Lấy ví dụ minh họa CSDL $\mathcal{D}$ ở Hình 2.1 với TID 101 chứa các mục: $\{1, 2, 3, 4\}$. Ngưỡng $minsup = 2$.

- Ở bước $k = 1$, $\overline{C}_1$ sẽ là: $\langle$ 101, $\{\{1\}, \{2\}, \{3\}, \{4\}\}$ $\rangle$.
- Ở bước $k = 2$, tập ứng viên C_2 chứa $\{1, 2\}$, $\{1, 4\}$, $\{1, 5\}$,... (lưu ý mục $\{3\}$ không đạt minsup ở L_1 nên các tập chứa $\{3\}$ không xuất hiện trong C_2). Vì TID 101 chứa các tổ hợp con hợp lệ nằm trong C_2 là $\{1, 2\}$, $\{1, 4\}$, $\{2, 4\}$, nên $\overline{C}_2$ cho TID 101 sẽ là: $\langle$ 101, $\{\{1, 2\}, \{1, 4\}, \{2, 4\}\}$ $\rangle$.
- Ở bước $k = 3$, tập ứng viên C_3 là $\{\{1, 2, 4\}, \{1, 4, 5\}\}$. Giao dịch 101 chỉ chứa tổ hợp $\{\{1, 2, 4\}\}$ có mặt trong C_3, nên $\overline{C}_3$ cho TID 101 sẽ là: $\langle$ 101, $\{\{1, 2, 4\}\}$ $\rangle$.

- Ở bước $k = 4$, vì $L_3 = \{\{1, 2, 4\}, \{1, 4, 5\}\}$ không có hai phần tử nào trùng tiền tố độ dài 2, nên hàm apriori-gen trả về $C_4 = \emptyset$ và vòng lặp kết thúc.

1.2.3 Mã giả (Pseudocode)

Giải thích mã giả:

1. (1) Tìm tập hợp các tập phổ biến có kích thước bằng 1 (L_1);
2. (2) Khởi tạo $\overline{C}_1 = \mathcal{D}$. Mỗi item trong mỗi giao dịch được xem là một tập 1-itemset, tạo thành định dạng ⟨TID, tập các 1-itemset⟩;
3. (3) Bắt đầu vòng lặp tìm các tập phổ biến kích thước k ($k \geq 2$). Vòng lặp này sẽ tiếp tục cho đến khi $L_{k-1} = \emptyset$;
4. (4) Sinh ra tập các ứng viên kích thước k (C_k) từ tập phổ biến L_{k-1}. Hàm apriori-gen dùng để sinh ra tập các ứng viên k-itemset C_k từ tập phổ biến L_{k-1};
5. (5) Khởi tạo tập $\overline{C}_k$ rỗng;
6. (6 - 11) Duyệt qua $\overline{C}_{k-1}$ để xác định và tăng biến đếm độ hỗ trợ cho các ứng viên trong C_k thực sự xuất hiện trong mỗi giao dịch. Sau đó xây dựng $\overline{C}_k$, chỉ giữ lại những giao dịch có chứa ít nhất một ứng viên k-itemset;
7. (12) Lọc ra tập phổ biến L_k. Chỉ giữ lại những ứng viên trong C_k có count $\geq$ *minsup*;
8. (14) Hợp nhất toàn bộ các tập L_k là danh sách tất cả các itemset phổ biến trong CSDL.

Hàm `apriori-gen` nhận đầu vào là L_{k-1}, tức là tập hợp tất cả các tập large $(k-1)$-itemset, và trả về một tập hợp gồm các large k-itemset. Nó gồm hai bước: (1) **join** — hợp L_{k-1} với chính nó để sinh ra các tập ứng viên C_k; và (2) **prune** — tỉa tất cả các ứng viên $c \in C_k$ nếu tồn tại bất kỳ $(k-1)$-itemset nào của c không nằm trong L_{k-1} (xem thủ tục `has_infrequent_subset` trong Hình 1.1).

Lấy ví dụ CSDL $\mathcal{D}$ ở Hình 2.1 có L_2: $\{\{1\ 2\}, \{1\ 4\}, \{1\ 5\}, \{2\ 4\}, \{4\ 5\}\}$. Bước join ghép các cặp có cùng item đầu tiên và item thứ hai theo thứ tự tăng dần: $\{1\ 2\} \bowtie \{1\ 4\}$, $\{1\ 2\} \bowtie \{1\ 5\}$, $\{1\ 4\} \bowtie \{1\ 5\}$, sinh ra C_3: $\{\{1\ 2\ 4\}, \{1\ 2\ 5\}, \{1\ 4\ 5\}\}$. Bước prune xóa $\{1\ 2\ 5\}$ vì tập con $\{2\ 5\}$ không nằm trong L_2. Do đó chỉ còn lại $\{\{1\ 2\ 4\}, \{1\ 4\ 5\}\}$ ở C_3.

1.2.4 Phân tích độ phức tạp

Phân rã chi phí theo từng bước

Bước 1 — Khởi tạo. Thuật toán quét toàn bộ CSDL $\mathcal{D}$ một lần duy nhất để tính độ hỗ trợ từng 1-itemset và khởi tạo $\overline{C}_1$. Độ phức tạp:

$$T_{\text{init}} = O(n \cdot w) \tag{1.5}$$

Bước 2 — Sinh ứng viên bằng `apriori-gen`. Hàm `apriori-gen` gồm hai pha con:

Thuật toán AprioriTID
Input:

- $\mathcal{D}$: cơ sở dữ liệu giao dịch;
- *minsup*: ngưỡng độ hỗ trợ tối thiểu.

Output: L, tập các itemset phổ biến trong $\mathcal{D}$.
Method:

(1) $L_1 \leftarrow$ {các 1-itemset phổ biến}
(2) $\overline{C}_1 \leftarrow \mathcal{D}$
(3) **for** $k \leftarrow 2$; $L_{k-1} \neq \emptyset$; k++ **do**
(4) $\quad C_k \leftarrow$ apriori-gen(L_{k-1}) ▹ sinh ứng viên mới
(5) $\quad \overline{C}_k \leftarrow \emptyset$
(6) $\quad$ **for all** phần tử $t \in \overline{C}_{k-1}$ **do**
(7) $\quad\quad C_t \leftarrow \{c \in C_k \mid (c-c[k]) \in t.\text{set-of-itemsets} \wedge (c-c[k-1]) \in t.\text{set-of-itemsets}\}$
(8) $\quad\quad$ **for all** ứng viên $c \in C_t$ **do**
(9) $\quad\quad\quad c.\text{count} \leftarrow c.\text{count} + 1$
(10) $\quad\quad$ **end for**
(11) $\quad\quad$ **if** $C_t \neq \emptyset$ **then**
(12) $\quad\quad\quad \overline{C}_k \leftarrow \overline{C}_k \cup \{\langle t.\text{TID}, C_t\rangle\}$
(13) $\quad\quad$ **end if**
(14) $\quad$ **end for**
(15) $\quad L_k \leftarrow \{c \in C_k \mid c.\text{count} \geq \mathit{minsup}\}$
(16) **end for**
(17) **return** $L = \bigcup_k L_k$

procedure apriori-gen(L_{k-1}: tập phổ biến $(k-1)$-itemset)**:**

(1) **for all** itemset $l_1 \in L_{k-1}$ **do**
(2) $\quad$ **for all** itemset $l_2 \in L_{k-1}$ **do**
(3) $\quad\quad$ **if** $(l_1[1] = l_2[1]) \wedge \ldots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$ **then**
(4) $\quad\quad\quad c \leftarrow l_1 \bowtie l_2$ ▹ bước join: sinh ứng viên
(5) $\quad\quad\quad$ **if** has_infrequent_subset(c, L_{k-1}) **then**
(6) $\quad\quad\quad\quad$ **delete** c ▹ bước prune: loại ứng viên không đạt
(7) $\quad\quad\quad$ **else**
(8) $\quad\quad\quad\quad$ thêm c vào C_k
(9) $\quad\quad\quad$ **end if**
(10) $\quad\quad$ **end if**
(11) $\quad$ **end for**
(12) **end for**
(13) **return** C_k

procedure has_infrequent_subset(c: ứng viên k-itemset; L_{k-1}: tập phổ biến $(k-1)$-itemset)**:**

(1) **for all** $(k-1)$-subset s của c **do**
(2) $\quad$ **if** $s \notin L_{k-1}$ **then**
(3) $\quad\quad$ **return** TRUE
(4) $\quad$ **end if**
(5) **end for**
(6) **return** FALSE

Hình 1.1: *Thuật toán AprioriTID tìm tập phổ biến dựa trên sinh ứng viên.*

Ký hiệu	Ý nghĩa
n	Số giao dịch trong CSDL $\mathcal{D}$
$\lvert I\rvert$	Số lượng hạng mục (item) phân biệt
w	Độ rộng trung bình của một giao dịch
$k_{\max}$	Độ dài tập phổ biến dài nhất
$\lvert C_k\rvert$	Số ứng viên ở bước k
$\lvert L_k\rvert$	Số tập phổ biến ở bước k
$\lvert\overline{C}_k\rvert$	Tổng số cặp $\langle$TID, itemset$\rangle$ trong $\overline{C}_k$
minsup	Ngưỡng độ hỗ trợ tối thiểu

Bảng 1.1: *Các ký hiệu dùng trong phân tích độ phức tạp.*

- *Join:* duyệt mọi cặp $(p, q) \in L_{k-1} \times L_{k-1}$, mỗi phép so sánh tốn $O(k)$ để kiểm tra $k-2$ item đầu khớp và item cuối theo thứ tự:

$$T_{\text{join}}(k) = O\big(|L_{k-1}|^2 \cdot k\big) \tag{1.6}$$

- *Prune:* với mỗi ứng viên $c \in C_k$, sinh k tập con kích thước $(k-1)$ và kiểm tra thành viên trong L_{k-1}:

$$T_{\text{prune}}(k) = O(|C_k| \cdot k \cdot |L_{k-1}|) \tag{1.7}$$

Tổng chi phí `apriori-gen` tại bước k:

$$T_{\text{gen}}(k) = O\big(|L_{k-1}|^2 \cdot k \;+\; |C_k| \cdot k \cdot |L_{k-1}|\big) \tag{1.8}$$

Bước 3 — Đếm support qua $\overline{C}_k$. Đây là điểm then chốt phân biệt AprioriTID với Apriori cơ bản: thay vì quét lại $\mathcal{D}$, thuật toán duyệt cấu trúc trung gian $\overline{C}_{k-1}$. Với mỗi phần tử $t \in \overline{C}_{k-1}$, thuật toán xác định tập ứng viên C_t có trong t, tăng c.count cho mọi $c \in C_t$, rồi ghi $\langle t.\text{TID}, C_t\rangle$ vào $\overline{C}_k$ nếu $C_t \neq \emptyset$. Chi phí:

$$T_{\text{count}}(k) = O\Big(|\overline{C}_k| \cdot k\Big) \tag{1.9}$$

Khi k tăng, $|\overline{C}_k|$ thường giảm nhanh do transaction reduction, nhỏ hơn rất nhiều so với chi phí $O(n \cdot |C_k|)$ nếu phải quét lại $\mathcal{D}$.

Tổng chi phí thời gian. Gộp cả ba bước:

$$T_{\text{total}} = O(n \cdot w) + \sum_{k=2}^{k_{\max}} \Big[|L_{k-1}|^2 \cdot k \;+\; |C_k| \cdot k \cdot |L_{k-1}| \;+\; |\overline{C}_k| \cdot k\Big]. \tag{1.10}$$

Độ phức tạp thời gian theo trường hợp

Trường hợp tốt nhất. Xảy ra khi dữ liệu thưa và *minsup* cao: $|L_k|$ nhỏ, $k_{\max}$ nhỏ, và $|\overline{C}_k|$ giảm nhanh sau $k = 2$. Khi đó chi phí $T_{\text{count}}(k)$ chiếm ưu thế nhưng $\sum_k |\overline{C}_k| \ll n \cdot k_{\max}$,

và thuật toán dừng sớm sau vài vòng lặp. Chi phí xấp xỉ:

$$T_{\text{best}} = O(n \cdot w + |L_1|^2 + |\overline{C}_2| \cdot 2) \tag{1.11}$$

AprioriTID vượt trội so với Apriori chuẩn nhờ tránh được $k_{\max}$ lần quét lại $\mathcal{D}$.

Trường hợp trung bình. Trên CSDL giao dịch thực tế điển hình, số tập phổ biến có phân bố thưa dần theo k: $|L_k|$ đạt cực đại quanh $k = 2$ hoặc $k = 3$ rồi giảm dần, và $|\overline{C}_k|$ co lại nhanh ở các bước sau. Chi phí bị chi phối bởi vài vòng lặp đầu tiên:

$$T_{\text{avg}} = O\left(n \cdot w + \sum_{k=2}^{k_{\max}} |\overline{C}_k| \cdot k\right) \tag{1.12}$$

với tổng $\sum_k |\overline{C}_k|$ thường chỉ là bội số hằng số của $n \cdot w$. Đây là chế độ hoạt động mà AprioriTID được thiết kế để tối ưu.

Trường hợp xấu nhất. Xảy ra khi dữ liệu **dày đặc** và *minsup* thấp: gần như mọi itemset đều phổ biến, $|L_k|$ tăng theo hướng tổ hợp $\binom{|I|}{k}$, và $|\overline{C}_k|$ không co lại. Khi đó số ứng viên tối đa là $O(2^{|I|})$ và độ phức tạp thời gian trở thành *hàm mũ* theo $|I|$:

$$T_{\text{worst}} = O\left(2^{|I|} \cdot |I|\right). \tag{1.13}$$

Đây là giới hạn lý thuyết chung cho mọi thuật toán sinh-và-kiểm-tra (*generate-and-test*) dựa trên Apriori.

Độ phức tạp không gian

Cấu trúc	Không gian	Ghi chú
CSDL gốc $\mathcal{D}$	$O(n \cdot w)$	Lưu trữ đầu vào
C_k, L_k mỗi vòng	$O(\lvert C_k\rvert \cdot k)$	Tập ứng viên và tập phổ biến
$\overline{C}_k$ (đặc trưng AprioriTID)	$O(n \cdot \lvert C_k\rvert)$	Lớn nhất ở $k = 2$, giảm dần

Bảng 1.2: *Độ phức tạp không gian của các cấu trúc chính.*

Điểm đáng chú ý là $\overline{C}_k$ ở giai đoạn $k = 2$ có thể **lớn hơn** $\mathcal{D}$ gốc: mỗi giao dịch w item ánh xạ thành $\binom{w}{2}$ cặp ứng viên, dẫn tới tăng *bậc hai* theo w. Đây là điểm yếu bộ nhớ chính của AprioriTID so với Apriori tiêu chuẩn, đặc biệt rõ trên dữ liệu dày đặc (xem minh họa ở Chương 2).

Các yếu tố ảnh hưởng đến hiệu năng

Ba yếu tố chính quyết định hiệu năng thực tế của AprioriTID:

- **Ngưỡng *minsup*.** Yếu tố ảnh hưởng mạnh nhất. *minsup* thấp làm $|L_k|$, $|C_k|$ và $|\overline{C}_k|$ tăng nhanh theo cấp số, kéo theo cả chi phí thời gian lẫn bộ nhớ. Ngược lại,

minsup cao giúp transaction reduction cắt tỉa $\overline{C}_k$ nhanh chóng, đưa thuật toán về gần trường hợp tốt nhất.

- **Kích thước và mật độ của CSDL.** Số giao dịch n chỉ ảnh hưởng tuyến tính ở bước khởi tạo $O(n \cdot w)$. Ngược lại, *mật độ* của dữ liệu (tỷ lệ item mỗi giao dịch chứa) mới là yếu tố then chốt: dữ liệu dày đặc (chess, mushroom) làm $|\overline{C}_2|$ phình to và $k_{\max}$ dài hơn, còn dữ liệu thưa (retail, T10I4D100K) giúp thuật toán hội tụ nhanh.
- **Độ rộng trung bình giao dịch w.** Ảnh hưởng *bậc hai* lên kích thước $\overline{C}_2$ (mỗi giao dịch sinh $\binom{w}{2}$ cặp ứng viên) và ảnh hưởng tuyến tính lên bước khởi tạo. Giao dịch càng dài thì cả thời gian lẫn bộ nhớ cao điểm đều tăng.

Ba yếu tố này sẽ được kiểm chứng bằng thực nghiệm trong Chương 3.

1.2.5 So sánh lịch sử

Trong giai đoạn đầu của lĩnh vực khai phá tập phổ biến (FIM), các thuật toán tiên phong như AIS và SETM gặp nhiều hạn chế do phải sinh ứng viên "on-the-fly"(trên đường bay) trong quá trình đọc giao dịch và đòi hỏi việc rà quét, sắp xếp cơ sở dữ liệu gốc liên tục rất tốn kém. Sự ra đời của thuật toán Apriori và AprioriTID vào năm 1994 bởi R. Agrawal và R. Srikant đã tạo ra một bước ngoặt lớn khi giải quyết được những nhược điểm này. Điểm cải tiến mang tính lịch sử của AprioriTID là nó áp dụng hàm sinh ứng viên tiên nghiệm giúp thu hẹp không gian tìm kiếm, đồng thời sử dụng cấu trúc dữ liệu trung gian mã hóa ($\overline{C}_k$) để đếm độ hỗ trợ. Nhờ vậy, thuật toán không cần phải quét lại cơ sở dữ liệu gốc từ vòng lặp thứ hai trở đi, giúp giảm thiểu đáng kể chi phí truy xuất đĩa (I/O) so với các thuật toán tiền nhiệm.

Mặc dù là một bước tiến lớn, AprioriTID vẫn để lại một hạn chế cốt lõi: nó vẫn hoàn toàn phụ thuộc vào mô hình "sinh và kiểm tra"(generate-and-test), sinh ra vô số ứng viên nhỏ không cần thiết và dễ gây quá tải bộ nhớ khi xử lý các tập dữ liệu lớn. Để giải quyết bài toán này, các thế hệ thuật toán FIM sau đó đã thay đổi hoàn toàn cách tiếp cận. Nổi bật là thuật toán FP-Growth ra đời bằng cách áp dụng cấu trúc cây nén FP-tree kết hợp với chiến lược "chia để trị", giúp khai phá trực tiếp các tập phổ biến mà hoàn toàn không cần phải sinh ứng viên. Song song đó, thuật toán Eclat cũng xuất hiện, giải quyết điểm nghẽn bằng cách chuyển sang sử dụng định dạng dữ liệu dọc (vertical data format), cho phép tính toán độ hỗ trợ cực kỳ nhanh chóng và hiệu quả thông qua phép giao các tập mã giao dịch (TID_set).

2

Ví dụ minh họa

2.1 Ví dụ 1: Cơ sở

CSDL

TID	Items
101	1 2 3 4
102	1 2
103	1 4 5
104	1 2 4 5
105	5

$\overline{C}_1$

TID	Tập Itemset
101	{{1}, {2}, {3}, {4}}
102	{{1}, {2}}
103	{{1}, {4}, {5}}
104	{{1}, {2}, {4}, {5}}
105	{5}

L_1

Itemset	Support
{1}	4
{2}	3
{4}	3
{5}	3

C_2

Itemset
{1 2}
{1 4}
{1 5}
{2 4}
{2 5}
{4 5}

$\overline{C}_2$

TID	Tập Itemset
101	{{1 2}, {1 4}, {2 4}}
102	{{1 2}}
103	{{1 4}, {1 5}, {4 5}}
104	{{1 2}, {1 4}, {1 5}, {2 4}, {2 5}, {4 5}}

L_2

Itemset	Support
{1 2}	3
{1 4}	3
{1 5}	2
{2 4}	2
{4 5}	2

C_3

Itemset
{1 2 4}
{1 2 5}
{1 4 5}
{2 4 5}

$\overline{C}_3$

TID	Tập Itemset
101	{{1 2 4}}
103	{{1 4 5}}
104	{{1 2 4}, {1 2 5}, {1 4 5}, {2 4 5}}

L_3

Itemset	Support
{1 2 4}	2
{1 4 5}	2

Hình 2.1: *Ví dụ 1 — Các bước thực thi AprioriTID trên CSDL 5 giao dịch với minsup = 2.*

Dừng tại $k = 4$: Áp dụng *apriori-gen* trên $L_3 = \{\{1, 2, 4\}, \{1, 4, 5\}\}$, bước *join* yêu cầu hai itemset phải trùng $k - 2 = 2$ item đầu. Ở đây $\{1, 2, 4\}[1{:}2] = \{1, 2\}$ khác $\{1, 4, 5\}[1{:}2] = \{1, 4\}$, nên không sinh được ứng viên nào, tức $C_4 = \emptyset$. Thuật toán kết thúc.

Tập kết quả cuối cùng (với support tương ứng):

Itemset	Support	Itemset	Support
{1}	4	{1,5}	2
{2}	3	{2,4}	2
{4}	3	{4,5}	2
{5}	3	{1,2,4}	2
{1,2}	3	{1,4,5}	2
{1,4}	3		

Tổng cộng thu được **11 tập phổ biến**.

Kiểm tra chéo (cross-check) bằng liệt kê thủ công. Duyệt toàn bộ CSDL gốc:

- **1-itemset:** {1} xuất hiện trong T101, T102, T103, T104 ⇒ sup=4. {2} trong T101, T102, T104 ⇒ sup=3. {3} chỉ trong T101 ⇒ sup=1 (*loại*). {4} trong T101, T103, T104 ⇒ sup=3. {5} trong T103, T104, T105 ⇒ sup=3.
- **2-itemset phổ biến:** {1,2} (T101,T102,T104)=3; {1,4} (T101,T103,T104)=3; {1,5} (T103,T104)=2; {2,4} (T101,T104)=2; {4,5} (T103,T104)=2. Cặp {2,5} chỉ xuất hiện trong T104 nên bị loại.
- **3-itemset phổ biến:** {1,2,4} (T101,T104)=2; {1,4,5} (T103,T104)=2. Các tổ hợp khác hoặc chứa item không phổ biến, hoặc có support < 2.
- **4-itemset:** {1,2,4,5} chỉ xuất hiện trong T104 ⇒ sup=1 (*loại*).

Kết quả liệt kê thủ công đúng **11 tập phổ biến**, khớp hoàn toàn với đầu ra của thuật toán ở Hình 2.1.

2.2 Ví dụ 2: Tình huống đặc biệt

Điểm yếu nổi tiếng nhất của AprioriTID là ở giai đoạn $k = 2$, tập $\overline{C}_2$ có thể **lớn hơn rất nhiều** so với CSDL gốc $\mathcal{D}$. Điều này xảy ra khi dữ liệu dày đặc (dense): mỗi giao dịch chứa nhiều item, sinh ra số lượng cặp ứng viên rất lớn.

Ta thiết kế CSDL $\mathcal{D}'$ gồm 4 giao dịch, mỗi giao dịch chứa 6 item từ tập 7 item, với minsup = 2:

Database $\mathcal{D}'$

TID	Items
101	1 2 3 4 5 6
102	1 2 3 4 5 7
103	1 2 3 4 6 7
104	1 2 3 5 6 7

$\overline{C}_1$

TID	Tập Itemset
101	{{1},{2},{3},{4},{5},{6}}
102	{{1},{2},{3},{4},{5},{7}}
103	{{1},{2},{3},{4},{6},{7}}
104	{{1},{2},{3},{5},{6},{7}}

L_1

Itemset	Support
{1}	4
{2}	4
{3}	4
{4}	3
{5}	3
{6}	3
{7}	3

Hình 2.2: *Ví dụ 2 — Bước $k = 1$: Tất cả 7 item đều phổ biến.*

Phân tích kích thước $\overline{C}_1$: CSDL gốc $\mathcal{D}'$ có $4 \times 6 = 24$ mục (item entries). Tập $\overline{C}_1$ cũng

chứa đúng $4 \times 6 = 24$ mục (1-itemset entries) vì tất cả item đều phổ biến, không có item nào bị loại.

Bước $k = 2$: Do $|L_1| = 7$, hàm apriori-gen sinh ra $C_2 = \binom{7}{2} = 21$ ứng viên 2-itemset. Mỗi giao dịch chứa 6 item, tạo ra $\binom{6}{2} = 15$ cặp con hợp lệ. Tổng số mục trong $\overline{C}_2$:

$$|\overline{C}_2|_{\text{entries}} = 4 \times 15 = 60 \quad \text{vs} \quad |\mathcal{D}'|_{\text{entries}} = 24$$

$\overline{C}_2$ **lớn gấp 2.5 lần CSDL gốc!** Đây chính là điểm yếu bộ nhớ đặc trưng của AprioriTID trên dữ liệu dày đặc.

C_2 (21 ứng viên)

Itemset
{1 2}, {1 3}, {1 4}
{1 5}, {1 6}, {1 7}
{2 3}, {2 4}, {2 5}
{2 6}, {2 7}, {3 4}
{3 5}, {3 6}, {3 7}
{4 5}, {4 6}, {4 7}
{5 6}, {5 7}, {6 7}

$\overline{C}_2$ (**60 entries** — lớn hơn $\mathcal{D}'$)

TID	Tập Itemset (15 cặp mỗi giao dịch)
101	{{1 2},{1 3},{1 4},{1 5},{1 6},{2 3},{2 4}, {2 5},{2 6},{3 4},{3 5},{3 6},{4 5},{4 6},{5 6}}
102	{{1 2},{1 3},{1 4},{1 5},{1 7},{2 3},{2 4}, {2 5},{2 7},{3 4},{3 5},{3 7},{4 5},{4 7},{5 7}}
103	{{1 2},{1 3},{1 4},{1 6},{1 7},{2 3},{2 4}, {2 6},{2 7},{3 4},{3 6},{3 7},{4 6},{4 7},{6 7}}
104	{{1 2},{1 3},{1 5},{1 6},{1 7},{2 3},{2 5}, {2 6},{2 7},{3 5},{3 6},{3 7},{5 6},{5 7},{6 7}}

Hình 2.3: *Ví dụ 2 — Bước $k = 2$: $\overline{C}_2$ có 60 entries, gấp 2.5 lần CSDL gốc (24 entries).*

Đếm support từ $\overline{C}_2$, ta được L_2 gồm tất cả 21 cặp đều phổ biến (support ≥ 2):

- Support 4: {1 2}, {1 3}, {2 3}
- Support 3: {1 4}, {1 5}, {1 6}, {1 7}, {2 4}, {2 5}, {2 6}, {2 7}, {3 4}, {3 5}, {3 6}, {3 7}
- Support 2: {4 5}, {4 6}, {4 7}, {5 6}, {5 7}, {6 7}

Các bước tiếp theo ($k = 3, 4, 5$) tương tự. Tại $k = 3$: $|C_3| = \binom{7}{3} = 35$ ứng viên, $|L_3| = 31$ itemset phổ biến. Tại $k = 4$: $|L_4| = 22$. Tại $k = 5$: $|L_5| = 6$. Tổng cộng thu được $7 + 21 + 31 + 22 + 6 =$ **87 tập phổ biến**.

Tại sao đây là tình huống đặc biệt quan trọng?

- Với dữ liệu dày đặc (dense), mỗi giao dịch có w item sẽ sinh ra $\binom{w}{2}$ cặp trong $\overline{C}_2$. Khi w lớn, $\overline{C}_2$ tăng **bậc hai** theo w, trong khi $\mathcal{D}$ chỉ tăng tuyến tính.
- Đây là lý do chính khiến AprioriTID có thể tiêu tốn nhiều bộ nhớ hơn Apriori chuẩn (vốn quét lại $\mathcal{D}$ gốc), đặc biệt khi minsup thấp và dữ liệu dày đặc.
- Tuy nhiên, ở các bước k lớn hơn, $\overline{C}_k$ giảm nhanh do cơ chế transaction reduction — các giao dịch không chứa ứng viên bị loại bỏ. Lợi thế của AprioriTID thể hiện rõ ở các bước k cao, khi $\overline{C}_k \ll |\mathcal{D}|$.

Kiểm tra chéo: Liệt kê thủ công — mỗi giao dịch thiếu đúng 1 item (T101 thiếu 7, T102 thiếu 6, T103 thiếu 5, T104 thiếu 4). Một tập con S có support bằng $4 - |\{\text{item thiếu nằm trong } S\}|$. Bất kỳ tập con nào cũng xuất hiện trong ít nhất 2 giao dịch (vì chỉ có 4

giao dịch, mỗi giao dịch thiếu đúng 1 item khác nhau từ {4,5,6,7}). Riêng các tập con chỉ chứa {1,2,3} luôn có support = 4. Kết quả 87 tập phổ biến khớp với kết quả chạy thuật toán.

3

Thực nghiệm và đánh giá

Appendices